

Generative AI & LangChain Implementation

Detailed Revision Notes & Master Reference Sheet

Course: GenAI Full Course (Part 1)	Instructor: Sheryians AI School	Topic: LLMs, LangChain & Pydantic	Target Role: AI Engineer / Developer
---	--	--	---

1. Course Prerequisites & Series Roadmap

Essential Technical Prerequisites:

- **Python Programming:** Primary development language for implementing GenAI pipelines, SDKs, and backend services.
- **Machine Learning (ML) & Deep Learning (DL) Fundamentals:** Required for understanding model training, weights, neural architectures, and cracking AI technical interviews.
- **Transformers:** Core deep learning architecture powering modern GenAI and GPT (*Generative Pre-trained Transformer*) models.
- **Natural Language Processing (NLP):** Foundational text processing concepts including **Tokenization** and **Embeddings**.

4-Part GenAI Curriculum Roadmap:

Part	Module Topic	Key Concepts & Focus Areas
Part 1	LLM Foundations	LLM basics, API integration, LangChain components, Pydantic schemas
Part 2	RAG Systems	Retrieval-Augmented Generation, Vector DBs, document indexing, embeddings
Part 3	Agentic AI Systems	Autonomous Agents, tool calling, dynamic decision making, LangGraph
Part 4	Capstone & Deployment	End-to-end production AI app building, scaling, cloud deployment

2. The 4 Phases of GenAI Evolution

Phase	Layer Name	Description & Key Milestones
Phase 1	Research Layer	Academic paper publication proving neural concepts (e.g., LSTM, GRU, and Google Brain's landmark ' <i>Attention Is All You Need</i> ' paper establishing Transformers).
Phase 2	Bedrock Models	Tech giants (OpenAI, Google, Anthropic, xAI) leverage massive compute/GPU clusters to train base Foundation LLMs on global datasets (ChatGPT, Gemini, Claude).
Phase 3	Providers	Managed cloud platforms offering scalable API endpoints and keys for model access (OpenAI Platform, Google AI Studio, Groq API, Hugging Face).
Phase 4	AI Engineers	Developers consume provider APIs and orchestration frameworks (LangChain) to build real-world user applications.

3. What is a Large Language Model (LLM)?

An LLM is a deep neural network powered by the Transformer architecture that learns patterns from data:

- **Large:** Trained on massive datasets (books, websites, Wikipedia, code, conversations) requiring vast GPU clusters.

- **Language:** Understands and generates multilingual text (English, Hindi, French, Spanish, etc.) and code (Python, JS).
- **Model:** Neural network that predicts sequential text outputs based on learned mathematical patterns.

Core Concept — How LLMs Work: LLMs do not search Google live or 'think' like humans. Text is converted into numerical vectors (**Embeddings**). The model evaluates statistical patterns and sequentially predicts the **next most likely word** until output completion.

Model Taxonomy Comparison:

Category	Key Models / Ecosystem	Characteristics & Access Mode
Proprietary (Paid)	OpenAI (GPT-4/5) Google (Gemini) Anthropic (Claude) xAI (Grok)	Closed weights, accessed via paid cloud API endpoints. Highest frontier capability and intelligence.
Open Source (Free)	Meta (Llama 3) Mistral AI DeepSeek (R1/V3)	Open weights downloadable from Hugging Face or hosted via free/low-cost API providers.

4. Why Use LangChain?

The Vendor Lock-in Problem: Every LLM provider (OpenAI, Google Gemini, Anthropic, Mistral) provides a completely different SDK syntax, initialization logic, and response structure. Writing provider-specific code locks you into one vendor and makes switching models difficult.

The LangChain Solution: LangChain provides a standardized wrapper interface across all providers. Swapping underlying models (e.g., from GPT-4 to Gemini 2.5 or Mistral) requires changing only 1 line of code while keeping all downstream chains, prompts, and memory structures identical.

5. The 6 Core Components of LangChain

#	Component	Functionality & Primary Use Cases
1	Models	Chat Models (text-in/out), Embedding Models (text-to-vector for search), and Multimodal Models (text + image/audio/video).
2	Prompts	Instructions given to LLMs: Simple Prompts, System+User Prompts, dynamic PromptTemplates, and Structured Prompts (JSON).
3	Chains (Runnables)	Sequencing multiple operational steps into a single workflow (e.g., Summarize → Translate → Format).
4	Memory	Manages conversation history state across turns using explicit message roles (SystemMessage, HumanMessage, AIMessage).
5	Indexes / RAG	Connects external private data (PDFs, SQL DBs, CSVs) to LLMs using Vector Stores and Document Loaders.
6	Agents	Autonomous systems that dynamically evaluate context and choose external Tools (Web Search, Calculator, APIs) to solve complex goals.

6. Environment Setup, UV & API Key Security

• **UV Package Manager:** An ultra-fast, Rust-based Python package manager operating **10–100x faster than standard pip**.

- Create venv: `uv venv venv` | Activate: `source venv/bin/activate` | Install: `uv pip install langchain`

• **Environment Security (.env):** API keys must never be hardcoded into source code. Store them in a `.env` file and load them via `python-dotenv`.

```
# .env File Configuration
OPENAI_API_KEY=sk-proj-...
GROQ_API_KEY=gsk-...
GOOGLE_API_KEY=AizaSy...
MISTRAL_API_KEY=xyz...
HUGGINGFACEHUB_ACCESS_TOKEN=hf_...
```

7. Model Initializations & Key Hyperparameters

Model Instantiation Approaches in LangChain:

```
from dotenv import load_dotenv
from langchain.chat_models import init_chat_model
from langchain_openai import ChatOpenAI

load_dotenv() # Load API keys from .env

# Approach 1: Universal Initializer
model = init_chat_model('gpt-4.1-nano', model_provider='openai')

# Approach 2: Provider Class
model_groq = ChatGroq(model='llama-3.3-70b-versatile', temperature=0.2)
response = model.invoke('Explain quantum computing in 2 sentences.')
print(response.content)
```

Key Hyperparameters:

Parameter	Range / Values	Impact & Recommended Use Cases
temperature	0.0 → 1.0	Controls output randomness. 0.0 : Deterministic, precise (math, code, extraction). 0.8–1.0 : Creative writing, brainstorming.
max_tokens	Integer (e.g., 256, 1024)	Restricts the maximum generated token count. Essential for capping API expenditure and preventing infinite loops.

8. Hugging Face & Local Model Execution

Hugging Face hosts over 330,000+ open-source models. LangChain supports two primary execution strategies:

1. **Hugging Face Cloud API (Inference Endpoint):** Calls cloud-hosted models via `HuggingFaceEndpoint` or `ChatHuggingFace`. Requires API key, zero local GPU usage.
2. **Local Execution (Pipeline):** Downloads model weights directly to disk via `HuggingFacePipeline.from_model_id()`.

```
from langchain_huggingface import ChatHuggingFace, HuggingFacePipeline

# Local model execution (downloads ~2.2GB model weights to local storage)
pipe = HuggingFacePipeline.from_model_id(
    model_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    task='text-generation',
    pipeline_kwargs={'max_new_tokens': 100}
)
local_llm = ChatHuggingFace(llm=pipe)
res = local_llm.invoke('What is data science?')
```

9. Embeddings & Vector Representations

Embeddings are dense numerical floating-point vectors representing semantic meanings of text. Text with similar meanings produce vectors close to each other in high-dimensional vector space.

```
from langchain_huggingface import HuggingFaceEmbeddings

# Free local embedding model (384-dimensional dense vectors)
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')

query_vector = embeddings.embed_query('What is AI?')
doc_vectors = embeddings.embed_documents(['AI is machine intelligence.', 'Python is a language.'])
print(f'Vector length: {len(query_vector)}') # 384
```

10. Chatbot Architecture & Role Separation

Passing raw unstructured strings into a chatbot causes context drift, token exhaustion, and loss of control. LangChain solves this via explicit **Role Separation** using message objects:

- **SystemMessage**: Defines assistant persona, operational rules, constraints, and tone.
- **HumanMessage**: Wraps user input queries.
- **AIMessage**: Wraps LLM generated outputs stored in conversation history.

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

messages = [
    SystemMessage(content='You are a helpful and concise AI computer science tutor.'),
    HumanMessage(content='What is Python?'),
    AIMessage(content='Python is a high-level interpreted programming language.'),
    HumanMessage(content='What was my first question?')
]
response = model.invoke(messages)
print(response.content)
```

11. Prompt Templates & Dynamic Prompts

PromptTemplates decouple static prompt text from dynamic runtime variables, creating reusable prompt pipelines.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are an expert movie data analyst. Extract key information from text.'),
    ('human', 'Analyze the following movie summary: {movie_text}')
])

formatted_prompt = prompt.invoke({'movie_text': 'Interstellar is a 2014 sci-fi film directed by Nolan.'})
response = model.invoke(formatted_prompt)
```

12. Structured Output Generation with Pydantic

The Backend Integration Challenge: Unstructured natural language generated by LLMs cannot be directly ingested into SQL databases, MongoDB, or REST APIs without risking parsing errors.

The Pydantic + Output Parser Solution:

1. Define a strict data model class inheriting from `pydantic.BaseModel` with explicit type annotations.
2. Create a `PydanticOutputParser(pydantic_object=Movie)` parser instance.
3. Inject `parser.get_format_instructions()` into the system prompt.
4. Parse raw response text using `parser.parse(response.content)` to obtain a verified Python/JSON object.

```
from pydantic import BaseModel, Field
from typing import List, Optional
from langchain_core.output_parsers import PydanticOutputParser

class MovieSchema(BaseModel):
    title: str = Field(description='Movie title')
    release_year: Optional[int] = Field(description='Year of release')
    genre: List[str] = Field(description='List of genres')
    director: Optional[str] = Field(description='Director name')
    cast: List[str] = Field(description='Main cast members')
    rating: Optional[float] = Field(description='IMDb or audience rating')
    summary: str = Field(description='Concise 1-sentence plot summary')

parser = PydanticOutputParser(pydantic_object=MovieSchema)
# Pass parser.get_format_instructions() inside prompt template
```

Summary & Key Takeaways for AI Engineers:

- **Standardization:** Use LangChain wrappers to isolate application code from provider SDK changes.
- **Predictability:** Always enforce Pydantic schemas for backend API and DB integration.
- **Security & Cost:** Use `.env` for key security, set `max_tokens` and temperature appropriately.